

Muhammad Shoaib Ahmed

AI Engineer | Python Backend | RAG and Multi-Agent Systems

Berlin, Germany | +44 7472 219595 (WhatsApp) | ishoaib452@gmail.com | linkedin.com/in/muhammad-shoaib-ahmed-45bb5b167 | shoaibqbal452.github.io

Work authorisation: Berlin-based and authorised to work in Germany. I hold an approved German Chancenkarte, which converts to an EU Blue Card on a qualifying offer. No employer sponsorship process, no agency filings, no cost to the employer. Available immediately, and open to relocating anywhere in Germany.

PROFILE

Software engineer with 9 years of experience, now focused on taking LLM systems into production. Over the past two years I have shipped three production AI systems: a multi-agent RAG platform serving grounded, cited answers over 28,000+ documents, an internal meetings intelligence platform processing about 50 hours of calls a week into minutes and assigned tickets, and an AI newsroom with a story-level news recommendation engine over live-news APIs. Earlier career spans the BBC and product startups, from full-stack delivery through to AI systems architecture. I own what I ship end to end: Terraform-managed AWS infrastructure, CI/CD, monitoring and alerting, and the cost and latency of the pipelines behind it.

TECHNICAL SKILLS

Languages and frameworks: Python (7 yrs) | Django REST Framework (4) | FastAPI (3) | Node.js (3) | React / TypeScript (5) | Angular (3) | SQL (5) | REST APIs (7) | Microservices (5) | Pytest (5) | Jest (2) | Unit and integration testing

AI and LLM: Generative AI and LLM application development | LangGraph / LangChain (2) | RAG and vector databases: Pinecone, pgvector, text-embedding-3-small (2) | Prompt engineering | Agentic and multi-agent orchestration | RAG evaluation: golden-set harness, CI quality gates, online answer validation | LangSmith | OpenAI function calling | Whisper / speech-to-text | LLMops and MLOps practices for production AI services

Data and infrastructure: PostgreSQL (5) | Redis (5) | Celery (4) | AWS: ECS Fargate, ECR, ALB, CloudFront, S3, RDS, Route 53, ACM, SSM, Lambda (5) | GCP (2) | Supabase | Docker (5) | Kubernetes | CI/CD: GitHub Actions, Jenkins (5) | Git (9) | Linux

Infrastructure as code and operations: Terraform for AWS infrastructure | Containerised deployments and one-command releases | Feature flags and gradual rollout with LaunchDarkly | Observability: Datadog, AWS CloudWatch, LangSmith | Alerting, log and metric dashboards | Cost and latency tuning of LLM and data pipelines

Ways of working: Agile / Scrum | Sprint planning and daily standups | Code review | Technical mentoring | English-language international teams

WORK EXPERIENCE

Senior AI Engineer (Contract) | AI Labs

United Kingdom, Remote | 04/2026 to Present

Building production AI agents and agentic RAG systems across three deliveries: a public multi-agent RAG platform, an internal meetings intelligence platform, and a client AI newsroom.

thegoodscholar.com, multi-agent RAG platform (live in production)

- **Architecture:** Designed a stateful LangGraph multi-agent system over 28,793 curated texts. A Validator node gates unclear or out-of-scope questions before any retrieval, an 8-way LLM Router classifies intent and fans out to parallel retrieval across Pinecone namespaces, and a Synthesize node grounds every answer strictly in retrieved context and streams it token by token over Server-Sent Events.
- **Hallucination mitigation:** Built a self-correction loop where a dedicated Validate node downstream of synthesis checks generated answers against retrieved sources and triggers up to 2 retry passes on drift, catching unsupported answers inside the execution graph rather than after the fact.
- **Agentic tool use:** Built a streaming tool-calling loop where the model decides mid-answer to search a sandboxed knowledge base and continues the response using the results, surfaced inline as live tool status, with an end-to-end cancellable stop control.
- **Evaluation:** Built an offline golden-set eval harness wired into CI as a quality gate, alongside the online Validate node. The eval results pinpointed retrieval ranking as the next bottleneck, making a reranker the evidence-based next step rather than a guess.
- **Product features:** Real-time tools for prayer times, Hijri dates, Qibla direction, Sehri/Iftar and verse lookup; multi-turn context from the last 5 turns persisted in PostgreSQL; passwordless magic-link auth with JWT; every answer cites its sources.
- **Delivery:** Shipped the full stack: FastAPI on PostgreSQL (AWS RDS) with SQLAlchemy and JWT auth, a React, TypeScript and Vite streaming frontend, and AWS infrastructure across ECS Fargate, ECR, ALB, CloudFront, S3, Route 53, ACM and SSM, with one-command Docker deployments. Sustains ~10s average response time at ~3,000 tokens per query.
- **Infrastructure as code and operations:** Defined the AWS infrastructure in Terraform (ECS Fargate services, RDS, ALB, CloudFront, S3 and IAM) so environments are reproducible and reviewable rather than clicked together, and run the platform on CloudWatch logs, metrics and alarms.

Meetings intelligence platform (internal, company-wide)

- Live and used company-wide, processing about 50 hours of recorded calls every week.
- Built a multi-agent bot that joins Google Meet calls, records and transcribes in the language spoken, writes structured minutes, emails them to attendees, and then creates and assigns tickets from the decisions and action items in those minutes.
- Chose and built a self-hosted headless capture path (containerised Chromium plus ffmpeg) over a managed bot API, keeping recordings and transcripts inside company infrastructure on S3 rather than a third-party vendor.
- Integrated downstream into Jira, Linear, Slack and email so meeting output lands in the tools teams already work in, rather than in another document nobody opens.

AI newsroom platform and news recommendation engine (client delivery)

- **Ownership:** Own both the ingestion layer and the news recommendation engine for an AI newsroom that turns live-news API feeds into ranked stories, context and drafts, with human editorial approval before anything publishes.
- **Story clustering:** Every incoming item is embedded and joined to the nearest story within a rolling 48-hour window, or starts a new story. The engine ranks stories, not articles, so five sources covering one event surface as one pick.
- **Ranking without users:** Designed a base importance score from source diversity, arrival velocity, time-decayed freshness, provider urgency flags, source trust tiers and novelty against the archive, so the feed is useful on day one with no interaction data.

- **Cost-aware LLM use:** LLMs run at story level only (summaries and “what’s new” notes on updates), never across the whole feed, keeping cost and latency flat as volume grows. Editor actions (picked, dismissed, published) are logged from day one as a future re-ranking signal, with personalisation modelled as importance x affinity.
- **Ingestion at scale:** Scheduled ingestion across five live-news APIs sized for roughly 5,000 to 20,000 items a day, plus a six-month backfill of an estimated 1 to 4 million items designed to resume rather than restart on failure, into a durable, queryable archive. The same embeddings power related-story and trending-topic features.

Lead Engineer, AI and Backend | Hyperion

United States, Remote | 09/2024 to 03/2026

Crypto accounting and tax-compliance fintech. Promoted to lead after six months; led a team of 3 engineers and owned the platform’s AI work.

- **AI accounting agent:** Built a RAG pipeline with pgvector to ground LLM-based reconciliation decisions in each account’s transaction history and classification precedent, powering an agent that answers account-specific financial queries and detects discrepancies faster than manual review.
- **Trust and compliance:** Implemented confidence scoring on AI-generated classifications, routing low-confidence cases to human reviewers via a structured escalation path with full audit logging, validated at 90%+ production accuracy against ground-truth samples.
- **Data pipelines:** Designed Python ETL pipelines processing wallet transactions, asset data and real-time crypto pricing across multiple exchange integrations, scaling from 10,000 to over 250,000 transactions per wallet on a Django REST Framework and FastAPI microservices architecture.
- **Throughput:** Boosted transaction throughput 70% via Celery distributed tasks and Redis caching, and automated deployments through Docker, GCP and GitHub Actions, delivering tax-ready cost-basis and balance reports for enterprise compliance.
- **Infrastructure as code:** Managed the AWS side of the platform in Terraform, so infrastructure changes went through review with the code rather than by hand in the console.
- **Monitoring and incident response:** Ran the platform on Datadog and CloudWatch: dashboards and alerts on pipeline throughput, queue depth and error rates, so ingestion failures and slow reconciliation runs surfaced before clients noticed them.
- **Safe rollout:** Used LaunchDarkly feature flags to release changes gradually and keep a kill switch on risky paths, which mattered on a platform producing financial and tax reports.
- **Team leadership:** Led a team of 3 engineers: sprint planning, code review, architecture decisions and mentoring, with the AI and data platform work as my own delivery area.

Backend Integration Engineer | Hyperion

United States, Remote | 03/2024 to 08/2024

- Built and maintained the exchange and wallet integrations feeding the platform’s Python ETL pipelines, normalising blockchain and exchange data into structured financial records.

Full Stack Engineer (Contract) | BBC

London, United Kingdom | 06/2022 to 03/2024

BBC Bitesize (children’s education), part of the WebCore platform team.

- **Platform engineering:** Built reusable React components and containers for WebCore, BBC’s internal framework standardising UI and code reuse across broadcasting platforms, adopted by multiple BBC teams to cut duplicate frontend work.
- **Backend and delivery:** Developed backend microservices in Node.js and Django, built AWS Lambda functions for serverless logic, maintained Jenkins CI/CD pipelines and enforced quality through Jest testing and code review.
- **Operations:** Used Datadog dashboards and alerts to watch service health and track down issues on a product serving national-scale education traffic.

Full Stack Engineer | Vortechs Innovations (Civixa.ai)

Karachi, Pakistan | 07/2019 to 07/2022

Built Civixa.ai, a data annotation studio and document and data management portal, in a five-person team reporting to the CTO.

- **Annotation studio:** Built annotation features in Vue.js and React, plus mobile work in React Native.
- **Backend:** Developed backend features in Django and Node.js.
- **Infrastructure:** Dockerised the application and deployed it on EC2 behind Nginx with S3 for asset storage; automated deployments and managed the EC2 infrastructure. Maintained unit tests in Jest.

Full Stack Developer | Wukla / Paksign

Karachi, Pakistan | 06/2017 to 06/2019

Legal-tech sister companies: Wukla generates contracts and legal documents for non-lawyers; Paksign provides paperless contract execution through electronic signatures. Worked in a five-person team of engineers and the CTO. Started while completing BSc studies.

- **E-signature product:** Built the Paksign frontend in Angular 2, then upgraded it to Angular 4. Implemented client-side signature placement with server-side coordinate mapping, so signatures land accurately inside the final PDF.
- **Performance:** Re-architected PDF handling from processing whole documents to rendering page-level images, cutting processing load and speeding up the app.
- **Deployment and DevOps:** Deployed and ran the application on Amazon EC2 with Docker and Nginx, managed the EC2 instances, and automated releases with Jenkins pipelines.
- **Quality:** Tested the application with Selenium.

EDUCATION

Master’s Degree, Cyber Security (Merit) | Staffordshire University, United Kingdom | 01/2023 to 04/2024

Bachelor’s Degree, Computer Science | University of Bradford, United Kingdom | 2014 to 2018

LANGUAGES

English (C2) | Urdu (Mother tongue) | German (A1, actively learning)